

Pipeline Description

Overview

The deployment pipeline is configured in `.circleci/config.yml`. CircleCI is connected to the GitHub repository and runs the workflow when code is pushed to the `main` or `master` branch.

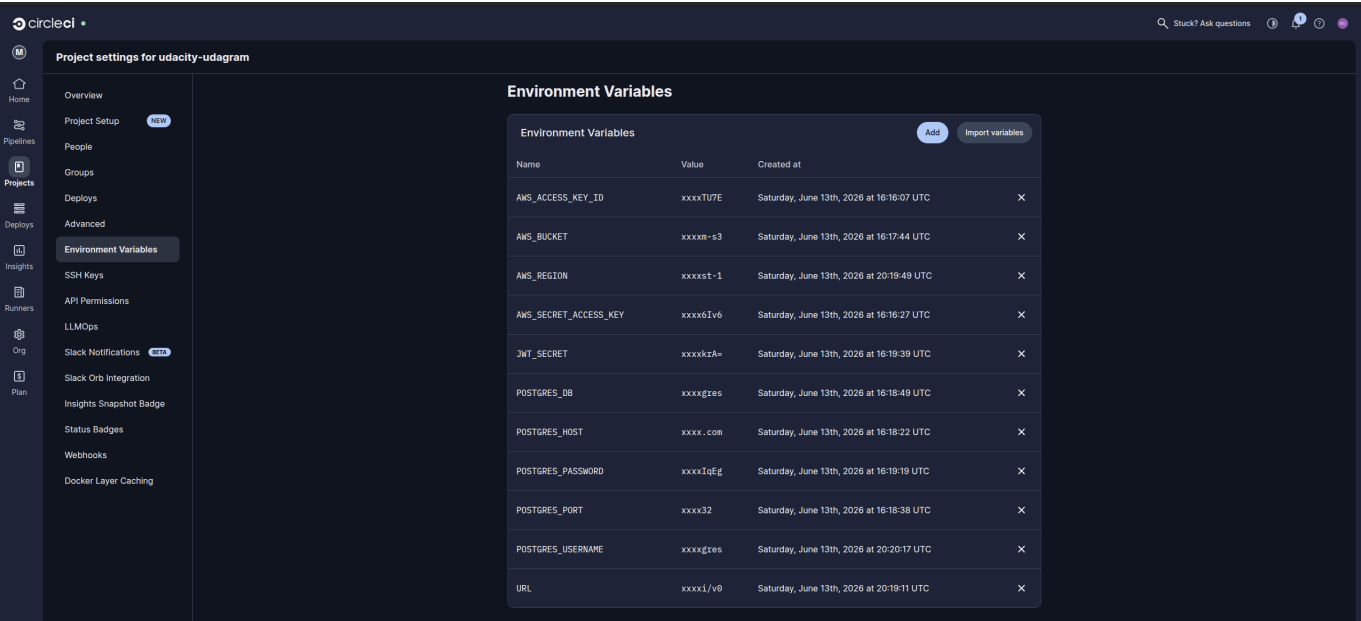
Pipeline Steps

- 1. Checkout source code from GitHub.
- 2. Install frontend dependencies with `npm run frontend:install`.
- 3. Install API dependencies with `npm run api:install`.
- 4. Run frontend linting with `npm run frontend:lint`.
- 5. Build the frontend with `npm run frontend:build`.
- 6. Build the API with `npm run api:build`.
- 7. Persist the frontend and API build outputs to the CircleCI workspace.
- 8. Wait for manual approval in the `hold` job.
- 9. Deploy the API to Elastic Beanstalk with `npm run api:deploy`.
- 10. Deploy the frontend build to S3 with `aws s3 sync`.
- 11. Configure the S3 bucket for static website hosting.

Pipeline Diagram

See [Pipeline.md](#).

CircleCI Screenshots



Wp6t5dmgMacuGKwWt1

U master

build-test-deploy #18

feat: add AWS_SESSION_TOKEN to required environment variables for deploymen...

Push by MariusJochheim 12m ago

Code source

Config source

Pipelines

Workflows

All Success Failed

udagram

build

hold

deploy

2m 05s

7s

3m 31s

udagram

build

hold

deploy

2m 05s

7s

03s

build

Duration 2m 03s

Executor Large

Steps Tests Timing Artifacts Resources

Spin up environment

2s

Preparing environment variables

0s

Install Node.js 14.15

5s

Checkout code

1s

Install Front-End Dependencies

28s

Install API Dependencies

14s

Front-End Lint

6s

Front-End Build

54s

API Build

13s

Persisting to workspace

0s

circleci

M

Home

Pipelines

Projects

Deploys

Insights

Runners

Org

Plan

PREVIEW

Chunk sidecars

Chunk

WJp65b5mgAIActuQKxWBs1 / master / build-test-deploy #18

feat: add AWS_SESSION_TOKEN to required environment variables for deploymen...

Push by MariusJochheim 12m ago

Code source

Config source

Workflows

All 2 Success 1 Failed 1

udagram

build

hold

deploy

udagram

build

hold

deploy

hold

Approved by mje

circleci

M

Home

Pipelines

Projects

Deploys

Insights

Runners

Org

Plan

PREVIEW

Chunk sidecars

Chunk

WJp65b5mgAIActuQKxWBs1 / master / build-test-deploy #18

feat: add AWS_SESSION_TOKEN to required environment variables for deploymen...

Push by MariusJochheim 12m ago

Code source

Config source

Workflows

All 2 Success 1 Failed 1

udagram

build

hold

deploy

udagram

build

hold

deploy

deploy

Duration 3m 30s

Executor Large

Track this deployment on the Deploys dashboard. Configure deploy markers

Steps

Tests

Timing

Artifacts

Resources

Spin up environment

Preparing environment variables

Install Node.js 14.15

Configure AWS Region for Deployment Tools

Setting Up Elastic Beanstalk CLI

Install AWS CLI - latest

Configure AWS Access Key ID

Checkout code

Attaching workspace

Verify Required Deployment Environment Variables

Configure and Deploy API to ElasticBeanstalk

Deploy Frontend to S3

3 / 5

Secrets

Production secrets are configured outside the repository. CircleCI stores the required deployment and application secrets as project environment variables. During the deploy job, CircleCI validates that the required variables exist, uses the AWS credentials to deploy the backend bundle to Elastic Beanstalk, and runs `eb setenv` so the Elastic Beanstalk production environment receives the same runtime values that the backend reads from `udagram/udagram-api/src/config/config.ts`.

These values are not committed to the repository. The checked-in `udagram/set_env.sh` file is only a local development template.

Required CircleCI project environment variables:

- `AWS_ACCESS_KEY_ID`
- `AWS_SECRET_ACCESS_KEY`
- `AWS_SESSION_TOKEN`
- `AWS_REGION`
- `AWS_BUCKET`
- `POSTGRES_HOST`
- `POSTGRES_DB`
- `POSTGRES_USERNAME`
- `POSTGRES_PASSWORD`
- `JWT_SECRET`
- `URL`

This project uses temporary AWS credentials. `AWS_ACCESS_KEY_ID`, `AWS_SECRET_ACCESS_KEY`, and `AWS_SESSION_TOKEN` must all come from the same current AWS credentials session. If one value is old or copied from a different session, S3 pre-signed upload URLs fail with `InvalidToken`.

Copy the values into CircleCI exactly as plain values. Do not include `export`, quotes, trailing spaces, or line breaks. With temporary AWS credentials, `AWS_ACCESS_KEY_ID` usually starts with `ASIA`; if it starts with `AKIA`, it is usually a long-lived key and should not be combined with a session token.

Do not use `POSTGRES_USER` or `AWS_DEFAULT_REGION` as the project variable names for this application. The backend configuration expects `POSTGRES_USERNAME` and `AWS_REGION`, so CircleCI and Elastic Beanstalk must use those exact names.

The CircleCI AWS and Elastic Beanstalk tooling expects `AWS_DEFAULT_REGION` internally. The deploy job derives that value from `AWS_REGION` at runtime, so `AWS_DEFAULT_REGION` does not need to be stored as a separate CircleCI project secret.

The deploy job passes these backend runtime values to Elastic Beanstalk:

```
eb setenv \  
  POSTGRES_USERNAME="$POSTGRES_USERNAME" \  
  POSTGRES_PASSWORD="$POSTGRES_PASSWORD" \  
  POSTGRES_HOST="$POSTGRES_HOST" \  
  POSTGRES_DB="$POSTGRES_DB" \  
  JWT_SECRET="$JWT_SECRET" \  
  AWS_BUCKET="$AWS_BUCKET" \  

```

```
AWS_REGION="$AWS_REGION" \  
AWS_ACCESS_KEY_ID="$AWS_ACCESS_KEY_ID" \  
AWS_SECRET_ACCESS_KEY="$AWS_SECRET_ACCESS_KEY" \  
AWS_SESSION_TOKEN="$AWS_SESSION_TOKEN" \  
URL="$URL"
```

When AWS temporary credentials expire, update all three AWS credential values in CircleCI and redeploy so Elastic Beanstalk receives the refreshed session.

The deploy job validates the AWS credentials with `aws sts get-caller-identity` before running `eb setenv`. If that check fails with `The security token included in the request is invalid`, the CircleCI AWS credential values are expired, malformed, or not from the same session.